Proyecto final de Gráficos por Computadora: Software Renderer

Sistema solar explorable desde una nave, renderizado **íntegramente por software** en Rust. Raylib solo se usa para abrir la ventana, leer teclado/ratón y mostrar el framebuffer (más texto 2D del HUD). No se usa ninguna función 3D de Raylib (ni `Camera3D`, ni `DrawModel`, ni shaders de GPU).

Cómo ejecutarlo

Requiere Rust (edición 2024) y las dependencias de `raylib-rs` para tu sistema.

```
cargo run --release
```

Autoría

Proyecto individual: toda la implementación (pipeline, física de la nave, generación de planetas, shaders, modelos y documentación) fue realizada por una sola persona.

Controles

Tecla	Acción
W / S	Avanzar / retroceder (hacia donde mira la nave)
A / D	Desplazarse a los lados
Espacio / Ctrl izq.	Subir / bajar
Shift izq.	Turbo
← / →	Girar la vista
↑ / ↓	Mirar arriba / abajo
Q / E o rueda del ratón	Acercar / alejar la cámara de la nave (zoom)
C	Activar/desactivar el **descarte** fuera de la vista
L	Activar/desactivar los **niveles de detalle** (LOD)
P	Mostrar/ocultar los modelos de superficie
H	Mostrar/ocultar el HUD con contadores
K	Iniciar el **colapso del universo** (K otra vez lo cancela y restaura todo)

La nave acelera y frena de forma progresiva. La cámara es de tercera persona y rígida: se recalcula cada frame desde la posición y la orientación de la nave, sin suavizado.

Características del menú de puntos y dónde verlas

Requisitos base

- **Pipeline propio** (Model · View · Projection · Viewport, rasterización con baricéntricas, depth buffer): `src/matrix.rs`, `src/shaders.rs` (vertex shader), `src/triangle.rs`, `src/framebuffer.rs`.
- **Sol, nave modelada por nosotros y planetas** que orbitan y rotan con el **tiempo transcurrido** (`delta_time`), no con el número de cuadros: `src/main.rs` (`PLANETS`, `planet_position`, `planet_rotation`).
- **Acercamiento y alejamiento sin transición de escena**: se puede rodear cualquier planeta y verlo de cerca.
- **Shaders de fragmentos propios** aplicados a la escena: `fragment_shader` en `src/shaders.rs`.

Planetas (5, cada uno con identidad propia)

Terran (continentes y bosques), Desierto (bandas y rocas), Hielo (grietas y cristales), Gigante gaseoso (bandas animadas) y Lava (grietas incandescentes y espigas). Definidos en la tabla `PLANETS` de `src/main.rs`. Cada uno tiene su propio shader, relieve y modelos.

Superficie

- **4 tipos de modelo propios** (árbol, roca, cristal, espiga volcánica), construidos proceduralmente: `src/surface.rs` (`build_prop`).
- **Distribución procedural reproducible** con ruido OpenSimplex y semilla fija (`PROP_SEED` en `src/main.rs`): misma semilla → mismas posiciones. Los grupos (bosques, campos de cristales) salen del umbral del ruido y no se solapan: `place_props`.
- **Posición y orientación correctas sobre la superficie curva mientras el planeta rota**: cada instancia usa la normal del terreno y se transforma con la misma rotación del planeta (`draw_props`).

Renderizado

- **Shaders con efecto distinguible y técnica distinta** (`src/shaders.rs`): cada planeta usa un método propio, no solo otro color.
  - Terran: biomas por altura (mar, playa, pasto, bosque, roca, nieve), casquetes polares y nubes.
  - Desierto: estratos de arena con ruido "ridged" y crestas de duna.
  - Hielo: placas de Voronoi con grietas y destellos.
  - Gigante gaseoso: bandas turbulentas analíticas y una gran tormenta en espiral.
  - Lava: corteza de Voronoi con grietas incandescentes emisivas.
  - Además: sol (emisivo), cristales y la nave.
- **Shaders animados** (cambian con el tiempo): nubes de Terran, destellos de Hielo, bandas y tormenta del gas, grietas de lava, sol y cristales.
- **Estilo visual** (todo en el fragment shader): *cel shading* por escalones de luz, puntos de semitono en las sombras (inspirado en Spider-Verse) y halo de atmósfera en el borde. Terran y Desierto tienen un acabado suave tipo Astroneer; Hielo, Gas y Lava, un acabado de cómic.
- **Mapeo de normales por software**: mapa de normales procedural (`src/normal_map.rs`) muestreado con interpolación bilineal; el fragment shader perturba la normal en el espacio tangente y recalcula la luz (`normal_map_ratio` en `src/shaders.rs`). Se puede notar el relieve fino al acercarse a Terran, Desierto, Hielo y Lava.
- **Deformación de la geometría con ruido**: los vértices de las icosferas se desplazan con ruido fractal (fbm de OpenSimplex): `TerrainGen` en `src/planet_mesh.rs`.

- **Etapa propia de generación y subdivisión de geometría**: icosaedro subdividido recursivamente (cada triángulo → 4), proyectado a la esfera y deformado, con normales planas o suaves: `src/planet_mesh.rs` (`generate_planet_mesh`). Se ejecuta en el arranque, antes de rasterizar.

Navegación

- **Control local para rodear un planeta**: a menos de 5 radios, la nave viaja con el planeta, así el planeta queda quieto respecto a la nave y se puede orbitar sin perderlo.
- **Aceleración, frenado y giro progresivos**: empuje con rozamiento exponencial, giro con velocidad angular que se acerca a la deseada. Todo depende de `delta_time`, así que responde igual en equipos de distinta velocidad.
- **Atracción gravitatoria** del Sol y de los planetas, que modifica la trayectoria.
- **Colisión básica**: no se puede atravesar el Sol ni los planetas (incluido su relieve).

Rendimiento (con contadores en el HUD)

- **Descarte fuera de la vista**: cuerpos contra el frustum de la cámara y modelos de superficie contra el frustum y el horizonte del planeta (`sphere_visible`, `draw_props`). Tecla **C** para comparar.
- **Niveles de detalle**: tres mallas por planeta (icosfera subdividida 2, 3 y 4 veces = 320, 1280 y 5120 triángulos), elegidas por tamaño aparente (`draw_body`). Tecla **L** para comparar.
- El HUD muestra cuerpos y modelos dibujados/descartados y los **triángulos procesados** por cuadro. Para la demostración: apuntar a un planeta lejano, alternar **C** y **L** y observar cómo cambian los contadores y los FPS.

Escenario: colapso del universo (tecla K)

Una de las teorías sobre el fin del universo: todo cae hacia el Sol y explota. Los planetas se acercan en espiral y giran cada vez más rápido, se estrellan contra el Sol (con chispas) y este se calienta hasta ponerse blanco. Luego estalla con una onda expansiva y un destello, y todo reaparece desde el blanco, con la nave de vuelta en su posición inicial. Duración aproximada: 22 s de colapso, 5 s de explosión y 3 s de renacimiento. Código: `planet_position` (parámetro `collapse`) y la máquina de fases en `src/main.rs`.

Presentación

Estética inspirada en Astroneer y Spider-Verse: caras planas (low-poly), escalones de luz, semitono, halos de atmósfera y una paleta propia por planeta. La nave (estilo Astroneer, por piezas de distinto color) es de creación propia y se genera con `assets/models/generar_nave_obj.py`. Las estrellas giran con la cámara y el HUD es discreto.

Estructura del código

Archivo	Contenido
`src/main.rs`	Bucle principal, física de la nave, cámara, sistema solar, descarte, LOD y HUD
`src/planet_mesh.rs`	Icosfera, subdivisión y deformación con ruido
`src/normal_map.rs`	Mapa de normales procedural

Archivo	Contenido
`src/surface.rs`	Modelos de superficie y su distribución
`src/shaders.rs`	Vertex shader y fragment shaders
`src/triangle.rs`, `src/framebuffer.rs`	Rasterización, iluminación Lambert y depth buffer
`src/matrix.rs`, `src/camera.rs`	Matrices Model/View/Projection/Viewport
`src/obj.rs`	Carga del OBJ de la nave por piezas y centrado por la caja envolvente
`assets/models/generar_nave_obj.py`	Genera `nave_astroneer.obj` (sin necesitar Blender)
`assets/models/generar_nave_astroneer.py`	Alternativa: script de Blender para generar una nave similar
`docs/GUIA_PARA_EXPLICAR.md`	Guía para entender y explicar el funcionamiento del programa

Recursos externos y asistencia de IA

- Dependencias: `raylib` (ventana, entrada y texto 2D), `tobj` (lectura de OBJ) y `noise` (ruido OpenSimplex).
- Se usó asistencia de IA (Claude) para programar y depurar partes del proyecto. Todo el código fue revisado y puede explicarse durante la demostración.

Video de la demostración

El video se entrega junto con el repositorio. Muestra el recorrido completo: vuelo espacial, acercamiento a un planeta, vuelta alrededor de él y alejamiento.